CARD PARTNER

Una web para amantes de los TCGS

Servicios y Sistemas Web

Laboratorio L2
GRUPO 33

ÁLVAREZ FULCONIS, MARCOS
LÓPEZ PRIMO, ÁLVARO
OTERO GARCÍA, SERGIO
SANTIESTEBAN MENDO, EDGAR

Curso 25-26

Índice

1. Base de datos **1**
1.1. Cambios del Modelo de Dominio a Entidad-Relacion 1

2. Estructura de la Base de Datos **2**
2.1. Componentes Principales 2
2.2. Resumen de Relaciones y Cardinalidad 2
2.3. Inserción de los datos en la BBDD en Apache NetBeans 3
2.4. Inserción de los datos en las tablas 4
2.5. Consideraciones Técnicas 4

3. Problemática: CORS y Hotlinking **5**
3.1. Solución: Uso de un Proxy Inverso 5

4. Instrucciones de ejecución del código **5**

5. Reparto de carga de trabajo **7**

6. Declaración de Uso de Inteligencia Artificial **7**

1. Base de datos

Para la base de datos se ha elegido usar MariaDB a través de XAMPP para facilitar el trabajo de gestión y diseño, ya que permite usar phpMyAdmin desde navegadores web y gestionar la base de datos de manera visual y exportar o importar los scripts de creación e inserción de datos, además de otras herramientas útiles (como el meter csv).

1.1 Cambios del Modelo de Dominio a Entidad-Relacion

Para traducir el modelo de dominio realizado en la práctica 2 al modelo entidad-relacion de la base de datos, se han realizado los siguientes cambios:

- **Usuario**:
  Se ha traducido la relación de *Usuario* como clase padre de *Jugador* y *Admin* a una sola tabla `Usuario` y un atributo ROL que especifique que tipo de usuario es.

- **Mazo**:
  Al igual que con *Usuario*, la clase *Mazo* se ha traducido a una única tabla `Mazo` y un atributo ES_OFICIAL para hacer la distinción.

- **Relaciones *:***:
  Las relaciones muchos a muchos entre *Jugador*-**GUARDA**-*Mazo*, *Mazo*-**CONTIENE**-*Carta* y *Jugador*-**VALORA**-*Mazo* se han traducido como las clases intermedias `GUARDA_MAZO`, `CONTIENE_CARTA` y `VALORACION` respectivamente.

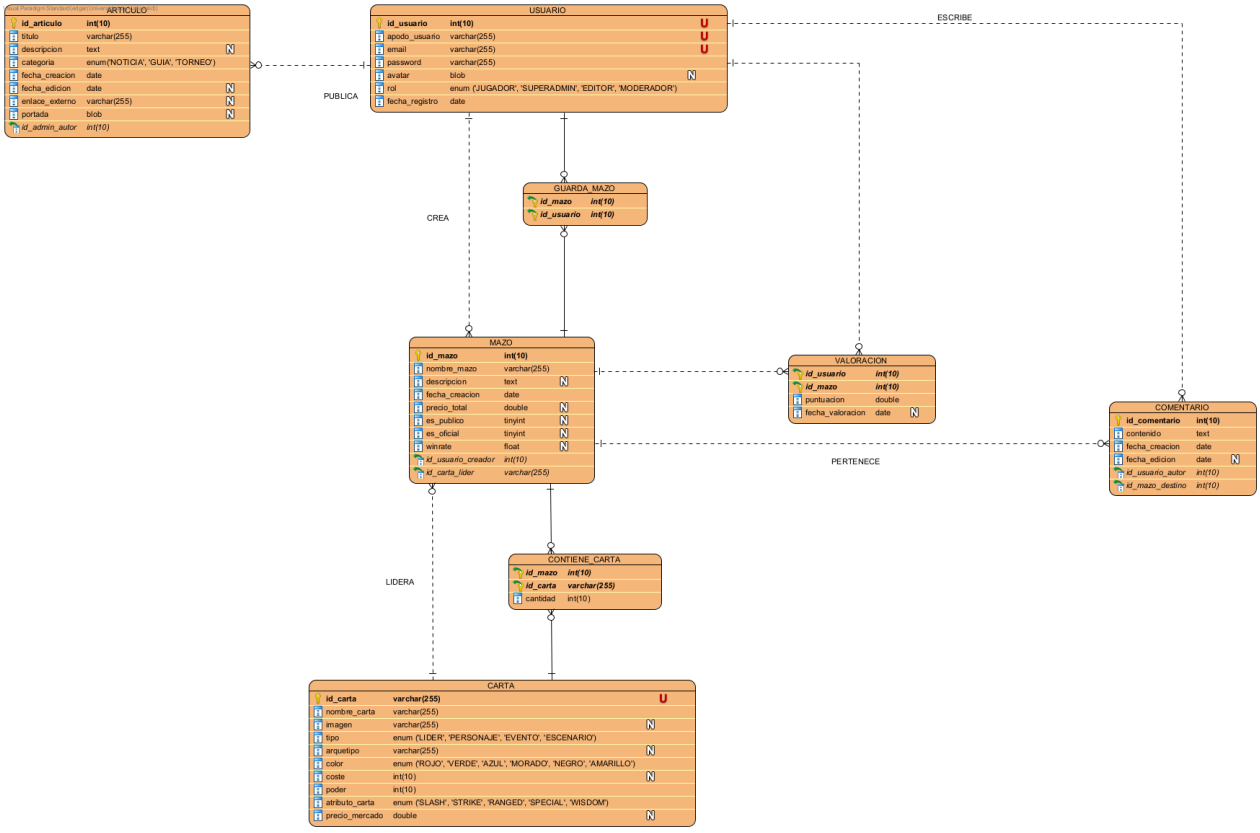


Figura 1: Entidad-Relacion de la base de datos

2. Estructura de la Base de Datos

La base de datos `cardpartner_db` está diseñada bajo un modelo relacional orientado a la gestión de una comunidad de juegos de cartas coleccionables (TCG). Su arquitectura se divide en cuatro pilares fundamentales:

2.1 Componentes Principales

- **Gestión de Usuarios y Roles:** La entidad `USUARIO` centraliza el acceso y la seguridad. Mediante un sistema de roles (*Jugador, Editor, Moderador, Superadmin*), se definen los niveles de interacción con la plataforma.
- **Catálogo de Cartas y Mazos:**
  - `CARTA`: Actúa como el diccionario técnico del juego, almacenando atributos estáticos como coste, poder, color y tipo.
  - `MAZO`: Representa la agrupación lógica de cartas creada por un usuario, vinculada obligatoriamente a una carta líder.
- **Módulo de Comunidad:** La interacción social se articula a través de las tablas `VALORACION`, `COMENTARIO` y `GUARDA_MAZO`, permitiendo la retroalimentación y la creación de bibliotecas personales.
- **Sistema de Contenidos:** La entidad `ARTICULO` permite a los administradores gestionar noticias, guías y torneos, manteniendo la plataforma actualizada.

2.2 Resumen de Relaciones y Cardinalidad

En esta sección comentamos las cardinalidades y relaciones en forma de tabla con una pequeña explicación de lo que significan para poder complementar el E/R.

Relación	**Tipo**	**Descripción**
Usuario → Mazo	1:N	Un usuario es autor de múltiples mazos.
Mazo ↔ Carta	N:M	Relación gestionada por `CONTIENE_CARTA` con cantidades.
Usuario ↔ Mazo	N:M	Biblioteca personal y favoritos (`GUARDA_MAZO`).
Usuario → Articulo	1:N	Relación de autoría para perfiles administrativos.

Cuadro 1: Resumen de la Arquitectura de Datos

2.3 Inserción de los datos en la BBDD en Apache NetBeans

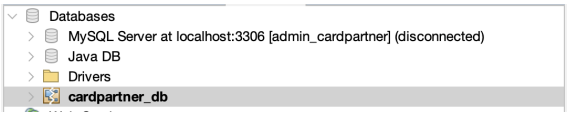


Figura 2: Hemos configurado la conexión en el Apache NetBeans con una base de datos MariaDB en el localhost con su puerto por defecto.

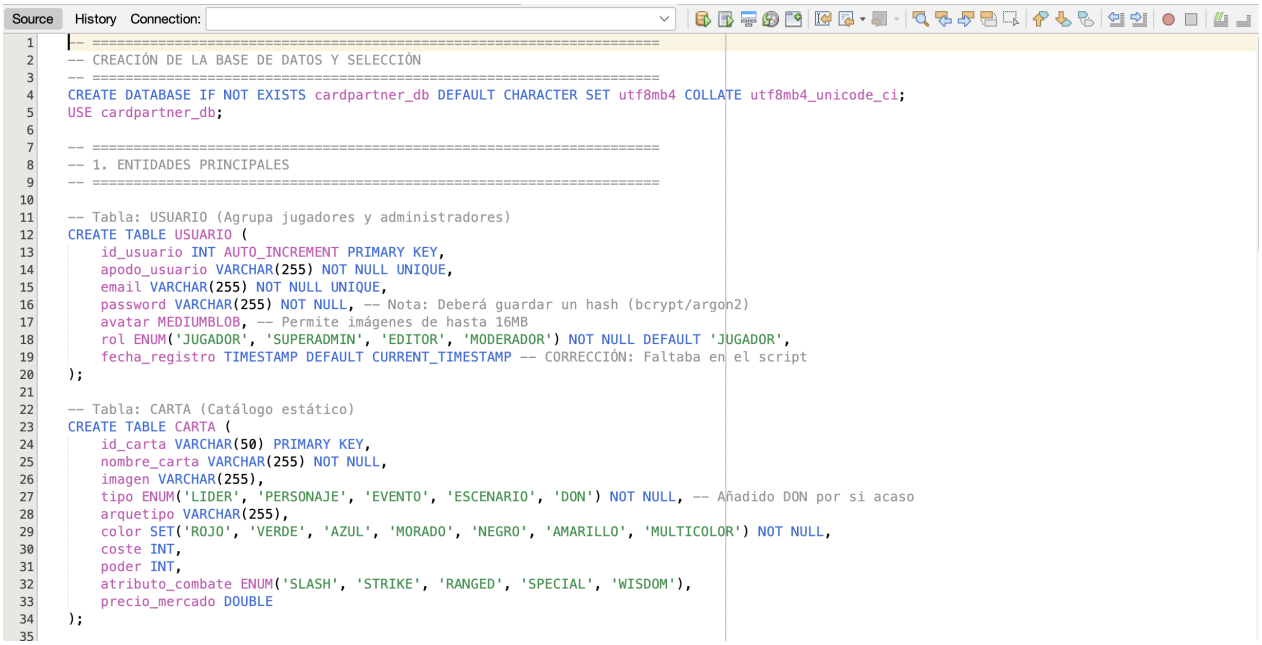


```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/CardPartner">
    <Resource
        auth="Container"
        driverClassName="org.mariadb.jdbc.Driver"
        factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
        initialSize="10"
        jdbcInterceptors="org.apache.tomcat.jdbc.pool.interceptor.ConnectionState;
      org.apache.tomcat.jdbc.pool.interceptor.StatementFinalizer"
        jmxEnabled="true"
        logAbandoned="true"
        maxActive="100" maxWait="10000"
        minEvictableIdleTimeMillis="30000"
        minIdle="10"
        name="jdbc/cardpartner_db"
        username="admin_cardpartner"
        password="123456"
        removeAbandoned="true"
        removeAbandonedTimeout="60"
        testOnBorrow="true"
        testOnReturn="false"
        testWhileIdle="true"
        timeBetweenEvictionRunsMillis="30000"
        type="javax.sql.DataSource"
        url="jdbc:mariadb://localhost:3306/cardpartner_db"

        validationInterval="30000"
        validationQuery="SELECT 1"/>
</Context>
```

Figura 3: Modificamos el *context.xml* del editor para poder establecer la conexión permanente con la bbdd en nuestro proyecto.

El problema que hemos tenido es que al establecer la conexión y hacer un *push* al repositorio, Edgar, no tenía la conexión establecedia en el Apache, tras una tutoría con el profesor del laboratio, nos ha explicado que como no contamos con un servidor donde alojar la base de datos, realmente no había ninguna manera de hacer esta conexión persistente, por lo que tomamos la decisión de hacer unos scripts, que hagan la creación de la base de datos en local más simple, facilitando así su posterior conexión con el editor de código.



```
Source  History  Connection:
-- ========================================================================
-- CREACIÓN DE LA BASE DE DATOS Y SELECCIÓN
-- ========================================================================
CREATE DATABASE IF NOT EXISTS cardpartner_db DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE cardpartner_db;

-- ========================================================================
-- 1. ENTIDADES PRINCIPALES
-- ========================================================================

-- Tabla: USUARIO (Agrupa jugadores y administradores)
CREATE TABLE USUARIO (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    apodo_usuario VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL, -- Nota: Deberá guardar un hash (bcrypt/argon2)
    avatar MEDIUMBLOB, -- Permite imágenes de hasta 16MB
    rol ENUM('JUGADOR', 'SUPERADMIN', 'EDITOR', 'MODERADOR') NOT NULL DEFAULT 'JUGADOR',
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- CORRECCIÓN: Faltaba en el script
);

-- Tabla: CARTA (Catálogo estático)
CREATE TABLE CARTA (
    id_carta VARCHAR(50) PRIMARY KEY,
    nombre_carta VARCHAR(255) NOT NULL,
    imagen VARCHAR(255),
    tipo ENUM('LIDER', 'PERSONAJE', 'EVENTO', 'ESCENARIO', 'DON') NOT NULL, -- Añadido DON por si acaso
    arquetipo VARCHAR(255),
    color SET('ROJO', 'VERDE', 'AZUL', 'MORADO', 'NEGRO', 'AMARILLO', 'MULTICOLOR') NOT NULL,
    coste INT,
    poder INT,
    atributo_combate ENUM('SLASH', 'STRIKE', 'RANGED', 'SPECIAL', 'WISDOM'),
    precio_mercado DOUBLE
);
```

Figura 4: Además, para que todos tengamos las mismas tablas, hemos incluido el script *CreacionDBv2.sql* para que genere automáticamente las tablas (por motivos de brevedad no hemos incluido todo el script ya que se puede consultar en *Files/sql* dentro del apache)

.

2.4 Inserción de los datos en las tablas

Para esta parte, debido al poco tiempo que hemos tenido para desarrollar la práctica por el calendario tan ajustado de entregas del curso, hemos decidido tener 2 scripts .sql para poblar la base de datos, los cuales se encuentran en *FIles/sql* dentro del Apache NetBeans.
Adicionalmente, para que fuese más cómodo y por un tema de estructura y revisión del código, hay una carpeta en nuestro repositorio de git (por ende en el zip en el que hemos entregado la práctica) llamado *DataBase* donde se encuentra el E/R, los scripts de creación de las tablas y los encargados de poblararla, así como los JSONs de los que hemos partidos para las cartas.
Queda pendiente para la siguiente entrega, modificar los DAOS y DTOs para que al emplear la librería java GSON, se encargue atuomáticamente de llenar la base de datos de cartas en función de un JSON, pero por motivos de coplejidad no hemos podido realizar dicha tarea para la actual entrega.

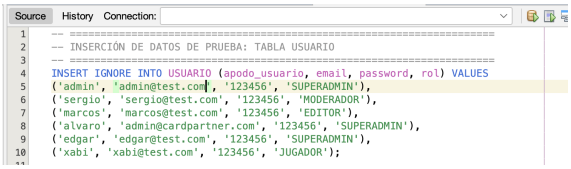


Figura 5: Hacemos un insert, con la peculiaridad de que usamos el modificador de sentencia *IGNORE* para que en caso de añadir más cartas, si encuentra la *primarykey* en la bbdd pasa de él.

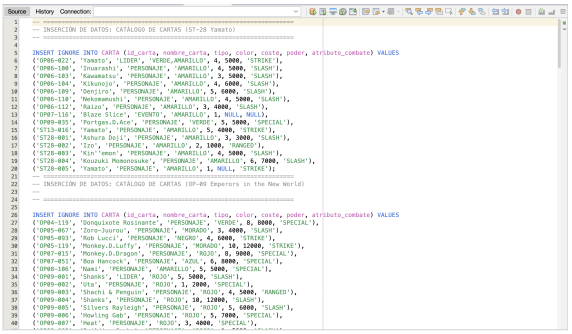


Figura 6: Script de población de la bbdd de cartas donde hemos insertado todas las cartas desde OP9 a OP12 además de un *starter deck* para que los compañeros de front tengan cartas de sobra. Al igual que en el insert anterior empleamos el modificador de sentencia *IGNORE*.

2.5 Consideraciones Técnicas

El diseño implementa integridad referencial mediante el uso de `ON DELETE CASCADE`, asegurando que la eliminación de una entidad principal (como un usuario) limpie automáticamente sus datos dependientes. Además, se han optimizado los tipos de datos utilizando `ENUM` para categorías fijas y `SET` para la gestión de colores múltiples en las cartas.

- **Gestión de Tiempos (*Timestamps* vs *Date*):** Se ha optado por el uso de `TIMESTAMP` en lugar de `DATE` para columnas como `fecha_registro` y `fecha_edicion`. Esto permite una precisión de segundos y, sobre todo, aprovecha la funcionalidad `DEFAULT CURRENT_TIMESTAMP` y `ON UPDATE CURRENT_TIMESTAMP`, automatizando el seguimiento de cambios sin necesidad de lógica adicional en el backend.
- **Uso de Tipos Enumerados (*ENUM* y *SET*):**
  - Se emplea `ENUM` para roles y categorías de artículos para restringir los valores a un conjunto cerrado, mejorando el rendimiento y evitando datos inconsistentes.
  - Se utiliza `SET` para el atributo `color` en las cartas, ya que permite que una sola carta pertenezca a múltiples categorías simultáneamente (cartas multicolores) de forma eficiente a nivel de bits.
- **Política de Nulidad (*Nullable*):** La mayoría de los campos críticos se han definido como `NOT NULL` para garantizar la calidad del dato. Los campos `NULL` (como `enlace_externo` o `portada`) se reservan exclusivamente para información opcional o metadatos que no interrumpen el flujo principal de la aplicación.

- **Almacenamiento de Binarios:** Para los avatares de usuario, se ha seleccionado `MEDIUMBLOB`. Aunque el estándar suele ser guardar rutas de archivos, incluir el binario (hasta 16MB) facilita la portabilidad inicial del proyecto y simplifica la gestión de copias de seguridad de la base de datos completa.
- **Claves Primarias y Foráneas:** Se utilizan `INT AUTO_INCREMENT` para entidades dinámicas (usuarios, mazos) para agilizar las búsquedas, mientras que las cartas utilizan un `VARCHAR(50)` como ID, permitiendo usar códigos alfanuméricos oficiales (ej: ÖP01-001¨) que facilitan la integración con APIs externas.

3. Problemática: CORS y Hotlinking

El problema principal se encuentra en dos conceptos bastante comunes de la seguridad web:

- **CORS (*Cross-Origin Resource Sharing*):** es un mecanismo de seguridad que impide que un navegador cargue recursos de un servidor, si esté no tiene permisos dentro del mismo.
- **Protección contra *Hotlinking*:** los servidores externos detectan la cabecera `Referer` de la petición, como en nuestro caso estamos intentando mostrar una imagen en un servidor que no es el oficial del TCG de One Piece, este bloquea la petición para ahorrar ancho de banda

3.1 Solución: Uso de un Proxy Inverso

Para solucionar esta limitación y evitarnos descargar miles de imágenes en nuestros pcs ()más luego subirlas a GitHub), hemos implementado un filtrado mediante el proxy **images.weserv.nl**.
Con esto lo que que hacemos es solicitar la imagen y el proxy, que tiene las cabeceras de CORS abiertas hace la petición a la web oficial del TCG de One Piece como si fuese un usuario y no otro sitio web y nos la envía para que la podamos mostrar.

La lógica implementada realiza la siguiente transformación de la URL original U, concatenándo la dirección del proxy con la dirección web donde se encuentra la carta:

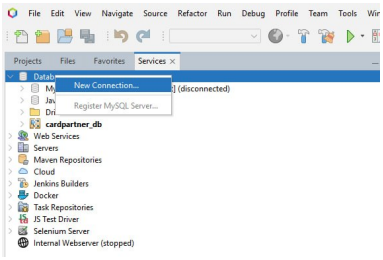
$$U_{final} = https://images.weserv.nl/?url = + U_{original} \tag{1}$$

Ventajas de la implementación

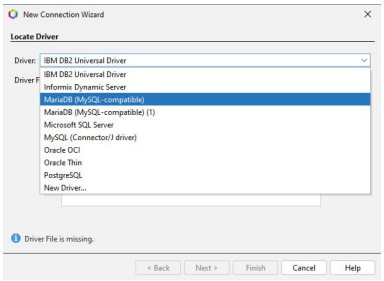
1. **Evasión de *Referer:*** el proxy actúa como intermediario. El servidor de origen ve la petición como si viniera del proxy y no de nuestra aplicación local, permitiendo la descarga, saltándonos ese bloque a peticiones desde webs de terceros por motivos de seguridad.
2. **Optimización:** el proxy permite redimensionar y cachear las imágenes, mejorando el tiempo de respuesta de nuestra interfaz, aunque es mucho más lento que sacar la imagen de una consulta a nuestra bbdd tenemos la ventaja de que este proceso es lento la primera vez, puesto que después la memoria caché hace que la latencia sea mínima, porque hemos limitado la cantidad de cartas que se muestran a la vez.
3. **Seguridad SSL:** convierte peticiones que podrían ser inseguras (`http`) a un entorno seguro (`https`), evitando errores de contenido mixto en el navegador.

4. Instrucciones de ejecución del código

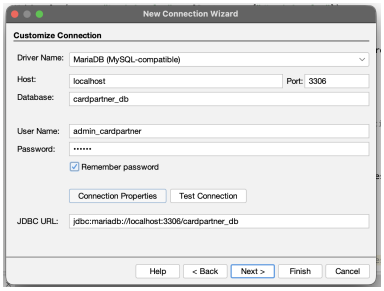
- Ejecutar el script de *setup_db* (que se encuentran en *CardPartner/sql/* o en el directorio raiz de SSW si se mira desde el repositorio de GitHub)
- Realizar la conexión con la bbdd en el Apache NetBeans



(a) Vamos al menú de *services* del editor



(b) En el menú desplegable seleccionamos nuestra base de datos y buscamos el driver que se encuentra en la carpeta lib de nuestro proyecto



(c) Introducimos el nombre de nuestra base de datos (cardpartner_db) con su usuario (admin_cardpartner) y la contraseña (123456)

- Vamos al menú de Files del editor
- Dentro de este menú, buscamos la carpeta que se llame sql
- Ejecutamos los scripts en orden para tener toda la información necesaria en nuestra base de datos.
- Ejecutamos la aplicación.

5. Reparto de carga de trabajo

Para esta tercera entrega, hemos decidido cambiar la manera de trabajar, nos hemos divido en dos grupos, Sergio y Marcos se han encargado del *frontend* mientras que Edgar y Álvaro se han encargado de la parte del *backend*.
Esta división se ha hecho con dos propositos, optimizar el tiempo de desarrollo,evitar conflictos en el git y permitir que los integrantes trabajen de la manera más cómoda posible.
Después de la segunda entrega, internamente vimos que Marcos y Sergio tenían más facilidad para el html y los estilos css por lo que se ofrecieron en encargarse de la parte más visual para seguir manteniendo una homogeneidad además de implementar las funcionalidades necesarias (los jsp). En el caso de ser necesario aportar conocimiento concreto sobre las cartas Marcos es un jugador medianamente especializado por lo que podría aportar la información necesaria.
Por otra parte, Edgar y Álvaro se encontraban más cómodos en la gestión de la bbdd, los daos,los dtos y *servlet*, por lo que eligieron de manera voluntaria trabajar en este campo de la web; además, como Álvaro es el que tiene más experiencia en este tcg, construir la base de datos, ha sido una tarea muy sencilla.
De esta manera, al estar en dos equipos, nos hemos podido organizar mejor minimizando así los posibles conflictos en el git puesto que al acotar las tareas, ambas personas del equipo podían trabajar sin pisar al otro integrante.

Nombre	**Porcentaje**	**Responsabilidad principal**
Equipo Frontend 50 %		
ÁLVAREZ FULCONIS, MARCOS P.	25 %	Todo lo relacionado con las vistas de cartas y las del apartado social, navegabilidad de las vistas, maquetación del documento
OTERO GARCÍA, SERGIO	25 %	Todo lo relacionado con las vistas de *decks* así como del login, register, página inicial de la web y redacción del documento.
Equipo Backend & Gestión 50 %		
LOPEZ PRIMO, ÁLVARO	25 %	Creación del esquema de la bbdd, colaborar en el E/R, población de la base de datos y transformación de los JSONs en datos para la misma, scripts para la automatización de MariaDB en local, redacción del documento.
SANTIESTEBAN MENDO, EDGAR	25 %	Colaboración en el E/R, creación de los esquemas de la bbdd, daos y dtos, redacción del documento.

Cuadro 2: Distribución de Responsabilidades por Equipo y Miembro

6. Declaración de Uso de Inteligencia Artificial

Para esta segunda entrega, se ha usado la IA Gemini como herramienta para dudas y mejoras del archivo LaTeX, revisión de los contenidos y evaluación de los mismos y diseño iterativo para mejorar lo ya realizado. Además, se ha usado Gemini para la generación del logo del proyecto.
Ejemplos de prompts realizados:

- 'Cómo funciona un serveles en java'
- 'Como hacer una conexión de MariaDB en ApacheNetBeans'
- 'Haciendo push, a mis compañeros no les aparece la bbdd ¿es normal?'
- 'Cómo puedo automatizar la instalación y creación de una bbdd con MariaDB'

- 'Me puedes revisar los *scripts* de creación de la base de datos'
- 'Traduceme este *script* de creación de bbdd para Windows en un .bat'
- 'Cómo puedo testear que la base de datos se ha rellenado de manera correcta con los *scrpts* sql'
- 'Cómo me puedo saltar la política CORS para hacer peticiones a la web de bandai nanco sobre cartas del tcg de One Piece'
- 'Cómo puedo cambiar la numeración de una página para sean botones y mejorar la navegabilidad por una lista de cartas'
- 'Ayudame a mejorar el botón de filtros de la vista cartas'
- '¿Cómo puedo añadir un mensaje de aviso que indique que no se puede acceder a esta vista sin estar loggeado?'
- 'No logro que se apliquen los filtros en las cartas, ¿puedes echarme una mano?'